

SV9 - Digital Verification using SV & UVM

====Assignment (4)====

Cross Coverage & Assertions

Name
Ibrahim Mohamed Mohamed Hanafy

Question1: ALSU-Revisited

-ALSU Design Code:

```
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON"; //to allow use of cin

input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction;
input [2:0] opcode;
input signed [2:0] A, B;

output reg [15:0] leds;
output reg signed [5:0] out;

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg;
reg signed [1:0] cin_reg; //cin is 1 bit<=====
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;

//Invalid handling=====
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] & opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;
```

```
always @(posedge clk or posedge rst) begin
    if(rst) begin
        cin_reg <= 0;
        red_op_B_reg <= 0;
        red_op_A_reg <= 0;
        bypass_B_reg <= 0;
        bypass_A_reg <= 0;
        direction_reg <= 0;
        serial_in_reg <= 0;
        opcode_reg <= 0;
        A_reg <= 0;
        B_reg <= 0;
    end else begin
        cin_reg <= cin;
        red_op_B_reg <= red_op_B;
        red_op_A_reg <= red_op_A;
        bypass_B_reg <= bypass_B;
        bypass_A_reg <= bypass_A;
        direction_reg <= direction;
        serial_in_reg <= serial_in;
        opcode_reg <= opcode;
        A_reg <= A;
        B_reg <= B;
    end
end
```

```
//leds output blinking=====
always @(posedge clk or posedge rst) begin
    if(rst) begin
        leds <= 0;
    end else begin
        if (invalid)
            leds <= ~leds;
        else
            leds <= 0;
    end
end
```

```

always @(posedge clk or posedge rst) begin
    if(rst) begin
        out <= 0;
    end
    else begin
        if (bypass_A_reg && bypass_B_reg)
            out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
        else if (bypass_A_reg)
            out <= A_reg;
        else if (bypass_B_reg)
            out <= B_reg;
        else if (invalid)
            out <= 0;
        else begin
            case (opcode) //Should use the registered opcode here<====
                3'h0: begin //OR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
                    else if (red_op_A_reg)
                        out <= |A_reg;
                    else if (red_op_B_reg)
                        out <= |B_reg;
                    else
                        out <= A_reg | B_reg;
                end
                3'h1: begin //XOR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
                    else if (red_op_A_reg)
                        out <= ^A_reg;
                    else if (red_op_B_reg)
                        out <= ^B_reg;
                    else
                        out <= A_reg ^ B_reg;
                end
                3'h2: out <= A_reg + B_reg; // cin wasnt taken into considerat
                3'h3: out <= A_reg * B_reg;
                3'h4: begin //SHIFT
                    if (direction_reg)
                        out <= {out[4:0], serial_in_reg};
                    else
                        out <= {serial_in_reg, out[5:1]};
                end
                3'h5: begin //ROTATE
                    if (direction_reg)
                        out <= {out[4:0], out[5]};
                    else
                        out <= {out[0], out[5:1]};
                end //3'h6 3'h7 should have been included<====
            endcase
        end
    end
end

```

-Bugs & Fixes

- Adder didn't implement cin where we check if FULL_ADDER is ON
- Registered opcode wasn't used in case block
- cin_reg size is wrong
- cases 3'h6 & 3'h7 weren't stated

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation, Then randomized in a loop and constrained to be off most of the time	-	A checker in the testbench to make sure the output is zero
Addition	If not Invalid: Addition should be correct with availability of carry cin if FULL_ADDER is ON	Randomization while constrained inputs : MAXPOS MAXNEG ZERO most of the time	Have max, min and zero corner bins to cover as well as a default bin	verified against the golden model
OR / XOR	When their opcode is set: If Reduction set: According to priority preform reduction operation	Randomized in a loop and constrained such that for reduction: the selected i/p has one bit high and the unselected one to have all low bits	Have all walking one values as bins to cover -Where the A bins are active if red_op_A set -And the A bins are active if red_op_B set and red_op_A low	verified against the golden model reduction should now always yield 1
OR / XOR	When their opcode is set: If NO reduction set: preform bitwise operation in i/p(s)	Randomized in a loop, no constraints	-	verified against the golden model reduction should now always yield 1
Addition / Multiplication	If not Invalid: Addition should be correct with availability of carry cin if FULL_ADDER is ON	Randomization while constrained inputs : MAXPOS MAXNEG ZERO most of the time	Have max, min and zero corner bins to cover as well as a default bin	verified against the golden model
Validity	-Opcode bits are set to 110 or 111 -red_op_A or red_op_B are high and the opcode is not OR or XOR operation Both yield invalid	Constrained such that invalid/ illegal cases are less frequent than valid ones	-	LED blinking functionality
bypass	The i/p is simply passed as o/p	Constrained such that bypasses aren't frequent	-	Check o/p vs i/p
Shift	Shift the output according to the direction input by 1 bit	Pure Random: No Constraints	-	verified against the golden model
Rotate	Rotate the output according to the direction input by 1 bit	Pure Random: No Constraints	-	verified against the golden model
Opcodes	according to each opcode preform the assigned function opcodes 6,7 aren't assigned to anything and considered invalid	Constrained ST: invalid opcodes are generated less often	Bins_shift[] Bins_arith[] Bins_bitwise[] illegal bins for invalid opcodes Transition bin of opcode form 0>1>....>5	Preform each function specified by the spec

- Package:

```
1 package class_pkg;
2
3     parameter INPUT_PRIORITY = "A";
4     parameter FULL_ADDER = "ON";
5
6     typedef enum reg [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
7     localparam MAXPOS = 3, ZERO = 0, MAXNEG = -4;
8
9
10    class RandomControl;
11        rand logic rst,red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in;
12        rand logic [2:0] opcode;
13        rand logic signed [2:0] A, B;
14
15        bit clk;
16        rand opcode_e my_opcodes[6];
17    endclass
```

```
//=====CONSTRAINTS=====
constraint Reset { //off for 90% of the time
    rst dist {0:/90 , 1:/10};
}

constraint Input { //MAXs and Zero 80% of the time for ADD and MULT
    if((opcode == ADD) | (opcode == MULT)){
        A dist {MAXPOS:=40,
            MAXNEG:=40,
            ZERO:=40,
            [-3:-1], [1:2] := 15};
        B dist {MAXPOS:=40,
            MAXNEG:=40,
            ZERO:=40,
            [-3:-1], [1:2] := 15}; //15/135
    }
}

constraint Bypass { //off for 90% of the time
    bypass_A dist {0:/90 , 1:/10};
    bypass_B dist {0:/90 , 1:/10};
}

constraint Validity { //Invalid for 10% of the time
    opcode dist {INVALID_6:/10,INVALID_7:/10,
        [OR:ROTATE]:/80};
    if(opcode == (OR | XOR)){
        red_op_A dist{0:/50 , 1:/50}; //both red high is unlikely?
        if(red_op_A){
            red_op_B dist{0:/80 , 1:/20};
        }
    }
}

constraint Reduction_Operations { //selected has one bit set (
    if(opcode inside {OR, XOR}){ //unslected has all low
        if(red_op_A){
            A dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
            B == 3'b000;
        }
        else if(red_op_B){
            B dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
            A == 3'b000;
        }
    }
}

constraint opcode_rand_array { //"constraint 8"
    unique {my_opcodes};
    foreach(my_opcodes[i]){
        my_opcodes[i] inside {[OR:ROTATE]};
    }
}
```

```
//=====COVER GROUPS=====
covergroup CovPort; //@(posedge clk)
  A_cp: coverpoint A iff(1)
  {
    bins A_data_0 = {ZERO};
    bins A_data_max = {MAXPOS};
    bins A_data_min = {MAXNEG};
    bins A_data_default = default;
    bins A_data_walkingones[] = {3'b001,3'b010,3'b100} i
  }

  B_cp: coverpoint B iff(1)
  {
    bins B_data_0 = {ZERO};
    bins B_data_max = {MAXPOS};
    bins B_data_min = {MAXNEG};
    bins B_data_default = default;
    bins B_data_walkingones[] = {3'b001,3'b010,3'b100} i
  }

  ALU_cp: coverpoint opcode iff(1)
  {
    bins Bins_shift[] = {SHIFT,ROTATE};
    bins Bins_arith[] = {ADD, MULT};
    bins Bins_bitwise[] = {OR, XOR};
    illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
    bins Bins_trans = (0 => 0 => 1 => 1 => 2 => 2 => 3 =
    //the transition sequence is like this as we wait tw
    //opcode in the testbench
  }
}
```

```
cin_cp: coverpoint cin iff(1){}
direction_cp: coverpoint direction iff(1){}
serial_in_cp: coverpoint serial_in iff(1){}
red_op_A_cp: coverpoint red_op_A iff(1){}
red_op_B_cp: coverpoint red_op_B iff(1){}
```

```
//=====Cross Coverage=====
```

```
//1.
cross ALU_cp,A_cp{
  bins Arith_corners_A_1 = binsof(ALU_cp.Bins_arith) &&
    binsof(A_cp.A_data_0);
  bins Arith_corners_A_2 = binsof(ALU_cp.Bins_arith) &&
    binsof(A_cp.A_data_max);
  bins Arith_corners_A_3 = binsof(ALU_cp.Bins_arith) &&
    binsof(A_cp.A_data_min);
  option.cross_auto_bin_max = 0;
}
```

```
cross ALU_cp,B_cp{
  bins Arith_corners_B_1 = binsof(ALU_cp.Bins_arith) &&
    binsof(B_cp.B_data_0);
  bins Arith_corners_B_2 = binsof(ALU_cp.Bins_arith) &&
    binsof(B_cp.B_data_max);
  bins Arith_corners_B_3 = binsof(ALU_cp.Bins_arith) &&
    binsof(B_cp.B_data_min);
  option.cross_auto_bin_max = 0;
}
```

```
//2.
cross ALU_cp,cin_cp{
  bins Add_cin = binsof(ALU_cp.Bins_arith[0]) &&
    binsof(cin_cp);
  option.cross_auto_bin_max = 0;
}
```

```
//3.
cross ALU_cp,direction_cp{
  bins Shifts_direction = binsof(ALU_cp.Bins_shift) &&
    binsof(direction_cp);
  option.cross_auto_bin_max = 0;
}
```

```
//4.
cross ALU_cp,serial_in_cp{
  bins Shift_Sin = binsof(ALU_cp.Bins_shift[0]) &&
    binsof(serial_in_cp);
  option.cross_auto_bin_max = 0;
}
//5.
cross ALU_cp,A_cp,red_op_A_cp,serial_in_cp
{
  bins bitwise_red_A = binsof(ALU_cp.Bins_bitwise) &&
    binsof(serial_in_cp) && binsof(red_op_A_cp) intersect{1};
  option.cross_auto_bin_max = 0;
}
//6.
cross ALU_cp,B_cp,red_op_B_cp,serial_in_cp
{
  bins bitwise_red_B = binsof(ALU_cp.Bins_bitwise) &&
    binsof(serial_in_cp) && binsof(red_op_B_cp) intersect{1};
  option.cross_auto_bin_max = 0;
}
//7.
cross ALU_cp,red_op_A_cp,red_op_B_cp //no OR,XOR bins and no transition bin
{
  ignore_bins red_at_no_bitwise = binsof(ALU_cp.Bins_bitwise) &&
    binsof(red_op_A_cp) intersect{0} &&
    binsof(red_op_B_cp) intersect{0};
  ignore_bins red_at_no_bitwise_2 = binsof(ALU_cp.Bins_trans);
  //option.cross_auto_bin_max = 0;
}
```

- Testbench:

```
reg signed [5:0] expected_out;
reg [15:0] expected_leds;

ALSU #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
) uut (.*);

ALSU_golden #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
) golden (.out(expected_out),.leds(expected_leds),.*
```

```
//Class and Objects
RandomControl cntrl = new();

//clk=====
initial begin
    clk = 0;
    forever begin
        #5 clk = ~clk;
        cntrl.clk = clk;
    end
end
```

```
//Sampling for Bins=====
always @(posedge clk)
    cntrl.CovPort.sample();
always @(posedge rst or posedge bypass_A or posedge bypass_B)
    cntrl.CovPort.stop();
always @(negedge rst or negedge bypass_A or negedge bypass_B) begin
    if(!rst && !bypass_A && !bypass_B)
        cntrl.CovPort.start();
    else
        cntrl.CovPort.stop();
end
```

```
task assert_reset;
    @(negedge clk);
    rst = 1;
    @(negedge clk);
    @(negedge clk);
    if (expected_out != 0) begin
        error_count = error_count + 1;
        $display("Error Resetting");
    end
    else
        correct_count = correct_count + 1;
    @(negedge clk);
    rst = 0;
endtask
```

```
task check_result;

    if((out == expected_out) && (leds == expected_leds))
        correct_count = correct_count + 1;
    else
        error_count = error_count + 1;

    if (out != expected_out) begin
        $display("=====");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b",
            opcode, bypass_A, bypass_B, red_op_A, red_op_B);
        $display("direction = %b | serial_in = %b", direction, serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected out = %d | Actual = %d", out, expected_out);
    end

    if (leds != expected_leds) begin
        $display("=====");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b",
            opcode, bypass_A, bypass_B, red_op_A, red_op_B);
        $display("direction = %b | serial_in = %b", direction, serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected leds = %d | Actual = %d", leds, expected_leds);
    end

endtask
```

```

initial begin

    correct_count = 0; error_count = 0;
    expected_out = 0; expected_leds = 0;
    $display("=====");

    //Test Series 1: Assert reset
    assert_reset();

    //Test Series 2: [First loop: constraint 8 is off]
    cntrl.opcode_rand_array.constraint_mode(0);
    repeat(1000) begin //take care when repeating for sequential
        assert(cntrl.randomize()); // as we check
    //use obejct's values
        A = cntrl.A;
        B = cntrl.B;
        opcode = opcode_e'(cntrl.opcode);

        cin = cntrl.cin; rst = cntrl.rst ;
        red_op_A = cntrl.red_op_A; red_op_B = cntrl.red_op_B;
        bypass_A = cntrl.bypass_A; bypass_B = cntrl.bypass_B;
        direction= cntrl.direction; serial_in = cntrl.serial_in;

    //wait for two clock cycles
        @(negedge clk);
        @(negedge clk);

        check_result();
        #1;
    end

```

```

//Test Series 3: [Second loop: constraint 8 only on, else di
//no bypasses, reduction or resetting
cntrl.constraint_mode(0);
cntrl.opcode_rand_array.constraint_mode(1);
repeat(3000) begin //take care when repeating for sequentia
    assert(cntrl.randomize()); // as we check
//use obejct's values
    A = cntrl.A;
    B = cntrl.B;

    cin = cntrl.cin; rst = 0;
    red_op_A = 0; red_op_B = 0;
    bypass_A = 0; bypass_B = 0;
    direction= cntrl.direction; serial_in = cntrl.serial_in;

    foreach(cntrl.my_opcodes[i])begin //6 iterations * 1000
        cntrl.opcode = cntrl.my_opcodes[i]; //all valid from
        opcode = opcode_e'(cntrl.opcode); //we didnt drive o
        //wait for two clock cycles
        @(negedge clk); //all 6 iterations share the same i/
        @(negedge clk);

        check_result();
        #1;
    end
end

```

- Do file

```
#vlib work
vlog -sv ../../../../VS_code/Session3/Assignment3/Q3_ALSU/* +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
#add wave *
coverage save ALSU_tb.ucdb -onexit
do wave.do
run -all

vcover report ALSU_tb.ucdb -details -all -output code_cvr.txt
```

-Code Coverage:

```
=====
=== File: ../../../../VS_code/Session3/Assignment3/Q3_ALSU/ALSU.v
=====
Statement Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                51         49         2      96.0

Branch Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Branches             34         32         2      94.1

***0***                3'h6: out <= 0;
***0***                3'h7: out <= 0;
```

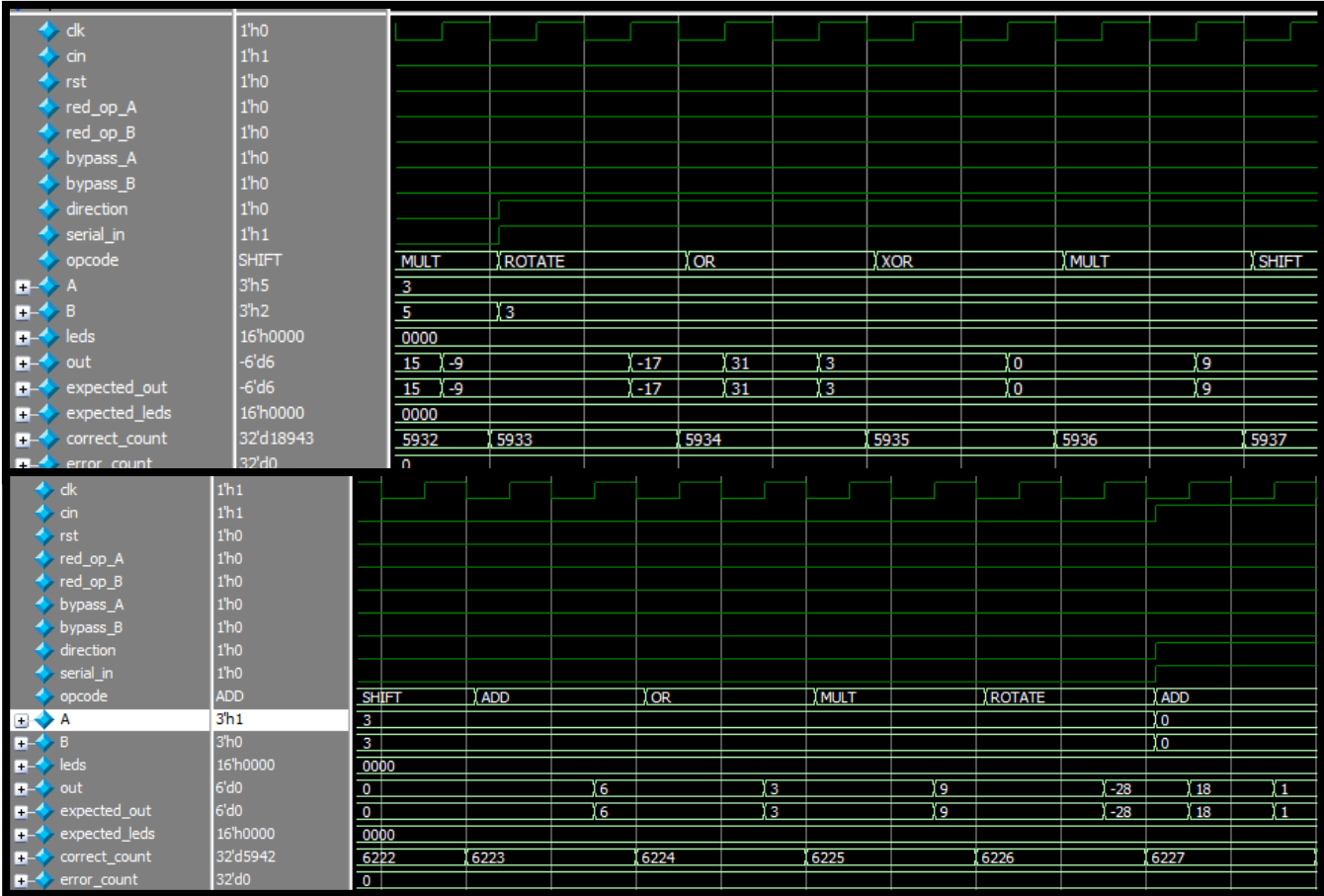
-The two invalid opcode (statements/Branches) as the design's code structure checks for invalidity before these statements

```
Toggle Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Toggle Bins          118        118         0     100.0
```

-Functional Coverage:

INST \class_pkg::RandomControl::Cov...	100.0%	100	100.0%		✓
CVP cin_cp	100.0%	100	100.0%		✓
CVP direction_cp	100.0%	100	100.0%		✓
CVP serial_in_cp	100.0%	100	100.0%		✓
CVP red_op_A_cp	100.0%	100	100.0%		✓
CVP red_op_B_cp	100.0%	100	100.0%		✓
CVP A_cp	100.0%	100	100.0%		✓
CVP B_cp	100.0%	100	100.0%		✓
CVP ALU_cp	100.0%	100	100.0%		✓
CROSS #cross_0#	100.0%	100	100.0%		✓
CROSS #cross_1#	100.0%	100	100.0%		✓
CROSS #cross_2#	100.0%	100	100.0%		✓
CROSS #cross_3#	100.0%	100	100.0%		✓
CROSS #cross_4#	100.0%	100	100.0%		✓
CROSS #cross_5#	100.0%	100	100.0%		✓
CROSS #cross_6#	100.0%	100	100.0%		✓
CROSS #cross_7#	100.0%	100	100.0%		✓

-Waveforms:



Question2: Assertions Examples

```
1  module Examples_SVA();
2
3  //1.
4  assert property (@(posedge clk) a |-> ##2 b);
5
6  //2.
7  assert property (@(posedge clk) (a&b) |-> ##[1:3] c);
8
9  //3.
10 sequence s11b;
11     req ##2 gnt;
12 endsequence
13 assert property (@(posedge clk) s11b |-> ##2 (!b));
14
15 //4.1
16 property p_decoder;
17     (@(posedge clk) $onehot(Y));
18 endproperty
19 assert_decoder: assert property (p_decoder);
20 //4.2
21 property p_encoder;
22     (@(posedge clk) ~(|D) |-> !valid);
23 endproperty
24 assert_encoder: assert property (p_encoder);
25
26 endmodule
```

Question3: Counter

-Counter Design Code:

```
 8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
22      always @(posedge clk or negedge rst_n) begin
23          if (!rst_n)
24              count_out <= 0;
25          else if (!load_n)
26              count_out <= data_load;
27          else if (ce)
28              if (up_down)
29                  count_out <= count_out + 1;
30              else
31                  count_out <= count_out - 1;
32          //missing else statement infers a latch
33          //which is actually the needed implementation
34      end
35
36      assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
37      assign zero = (count_out == 0)? 1:0;
38
39  endmodule
```

-No Bugs found

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation, then randomized such that it is off most of the time		A checker in the testbench to make sure the output is correct Can also check for zero flag ->assertion check:a_reset ->assertion check:a_zero
Load	When the load is asserted, the output count_out should take the value of the load_data input	Randomization, ST: It is active 70% of the time	Cover all load data values	A checker in the testbench to make sure the output is correct, matching the input ->Golden check ->assertion check:a_load,a_no_load
MAX	When max value is reached, the max_count signal is high	Randomization of enviroment	covered to be high at least once	->Golden Check->assertion check:a_max
Enabled1	When enabled, the counter counts up/down, count_out ouputs accordingly	Randomization, ST: It is active 70% of the time	Cover all count_out values from 0 to max twice: counting up & counting down	A checker to ensure counting every cycle ->assertion check:a_counting_up
Enabled2	When enabled, the counter counts up/down, and overflows correctly	Randomization, ST: It is active 70% of the time	Cover transition of overflowing twice: max=>0 (up) 0=>max (down)	A checker to ensure counting every cycle ->assertion check:a_counting_down

- Package:

```
class RandomControl #(parameter WIDTH = 4);
```

```
    parameter MAX = {WIDTH{1'b1}};
    rand logic up_down;
    logic [WIDTH-1:0] count_out;
    rand logic [WIDTH-1:0] data_load;
    bit clk;

    rand logic rst_n, load_n, ce;
```

```
//constraints=====
    constraint Reset { //off for 90% of
        rst_n dist {1:/90 , 0:/10};
    }
    constraint Load { //on for 70% of t
        load_n dist {0:/70 , 1:/30};
    }
    constraint Enable { //on for 70% of
        ce dist {1:/70 , 0:/30};
    }
```

```
covergroup CovPort @(posedge clk);
    Data_Load_cp: coverpoint data_load iff(rst_n && !load_n)
    {
        bins all_load_bins[] = {[0:$]};
    }

    counting_up_cp: coverpoint count_out iff(rst_n && ce && up_down)
    {
        bins all_count_up_bins[] = {[0:$]};
    }

    counting_up_of_cp: coverpoint count_out iff(rst_n && ce && up_down)
    {
        bins OF_max_to_zero = (MAX ==> 0);
    }

    counting_down_cp: coverpoint count_out iff(rst_n && ce && !up_down)
    {
        bins all_count_down_bins[] = {[0:$]};
    }

    counting_down_of_cp: coverpoint count_out iff(rst_n && ce && !up_down)
    {
        bins OF_zero_to_max = (0 ==> MAX);
    }
endcovergroup
```

- Testbench:

```
1  import class_pkg::*;
2  module Counter_tb #(parameter WIDTH = 4) (counter_if.TEST c_if);
3
4
5  RandomControl #(.WIDTH(WIDTH)) cntrl = new();
6
7
8  initial begin
9      if (!(WIDTH inside {4,6,8})) begin
10         $error("Invalid WIDTH=%0d. Allowed values are 4, 6, or 8", WIDTH);
11         $fatal;
12     end
13 end
14
15
16 initial begin
17     $display("=====TestBench Sta
18     assert_reset();
19     //Test Series: Randomization loop
20     repeat(1000) begin //take care when repeating for sequential testbenches
21         assert(cntrl.randomize()); // as we check and stimulate
22         //use obejct's values
23         c_if.rst_n = cntrl.rst_n;
24         c_if.load_n = cntrl.load_n;
25         c_if.data_load = cntrl.data_load;
26         c_if.up_down = cntrl.up_down;
27         c_if.ce = cntrl.ce;
28
29         @(negedge c_if.clk);
30         cntrl.count_out = c_if.count_out;
31         cntrl.CovPort.sample();
32         #1;
33     end
34     $display("=====TestBench Er
35     $stop;
36
37 end
```

```
task assert_reset;
    c_if.rst_n = 0;
    @(negedge c_if.clk);
    c_if.rst_n = 1;
endtask
```

-SVA module:

```
module counter_sva(counter_if.DUT c_if);

property p_load; //load has priority over enable
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (!c_if.load_n) | => (c_if.count_out == $past(c_if.data_load));
endproperty

property p_no_load;
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (c_if.load_n && ~c_if.ce) | => (c_if.count_out == $past(c_if.count_out));
endproperty

property p_counting_up;
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (c_if.load_n && c_if.ce && c_if.up_down) | => (c_if.count_out == $past(c_if.count_out) + 1'b1);
endproperty

property p_counting_down;
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (c_if.load_n && c_if.ce && !c_if.up_down) | => (c_if.count_out == $past(c_if.count_out) - 1'b1);
endproperty

property p_max;
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (c_if.count_out == {c_if.WIDTH{1'b1}}) | -> (c_if.max_count);
endproperty

property p_zero;
  @(posedge c_if.clk) disable iff (!c_if.rst_n)
    (c_if.count_out == {c_if.WIDTH{1'b0}}) | -> (c_if.zero);
endproperty

always_comb begin
  if(!c_if.rst_n)
    a_reset: assert final(c_if.count_out == 0);
end

a_load: assert property (p_load);
a_load_cover: cover property (p_load);

a_no_load: assert property (p_no_load);
a_no_load_cover: cover property (p_no_load);

a_counting_up: assert property (p_counting_up);
a_counting_up_cover: cover property (p_counting_up);

a_counting_down: assert property (p_counting_down);
a_counting_down_cover: cover property (p_counting_down);

a_max: assert property (p_max);
a_max_cover: cover property (p_max);

a_zero: assert property (p_zero);
a_zero_cover: cover property (p_zero);
```

-Wrapper:

```
module counter_wrapper(counter_if.DUT c_if);  
  
    counter dut (  
        .clk(c_if.clk),  
        .rst_n(c_if.rst_n),  
        .load_n(c_if.load_n),  
        .ce(c_if.ce),  
        .up_down(c_if.up_down),  
        .data_load(c_if.data_load),  
        .count_out(c_if.count_out),  
        .max_count(c_if.max_count),  
        .zero(c_if.zero)  
    );
```

-Top module:

```
module counter_top();  
    // 1. Generate the clock  
    bit clk;  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
    localparam WIDTH = 4;  
    // 2. instantiate the interface, and pass the clock  
    counter_if #(WIDTH) c_if(clk);  
    // 3. instantiate the tb, DUT, monitor, and pass the interface  
    counter_wrapper dut (c_if);  
    Counter_tb #(.WIDTH(WIDTH)) tb (c_if);  
    // 4. bind the SVA module to the design, and pass the interface  
    bind counter_wrapper counter_sva counter_sva_instance (c_if);  
  
endmodule
```

- Do file

```
#vlib work
vlog -sv ../../../../VS_code/Session4/Assignment4/Q3_Counter/* +cover -covercells
vsim -voptargs=+acc work.counter_top -cover
#add wave *
coverage save counter_top.ucdb -onexit
do wave.do
run -all

vcover report counter_top.ucdb -details -all -output code_cvr.txt
```

-Code Coverage:

```
=====
=== File: ../../../../VS_code/Session4/Assignment4/Q3_Counter/counter.v
=====
Statement Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                7         7         0     100.0

Branch Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Branches            10         10         0     100.0

Toggle Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Toggle Bins        30         30         0     100.0
```

-Functional Coverage:

[-]	INST \class_pkg::RandomControl...	100.0%	100	100.0%		✓
[+]	CVP Data_Load_cp	100.0%	100	100.0%		✓
[+]	CVP counting_up_cp	100.0%	100	100.0%		✓
[+]	CVP counting_up_of_cp	100.0%	100	100.0%		✓
[+]	CVP counting_down_cp	100.0%	100	100.0%		✓
[+]	CVP counting_down_of_cp	100.0%	100	100.0%		✓

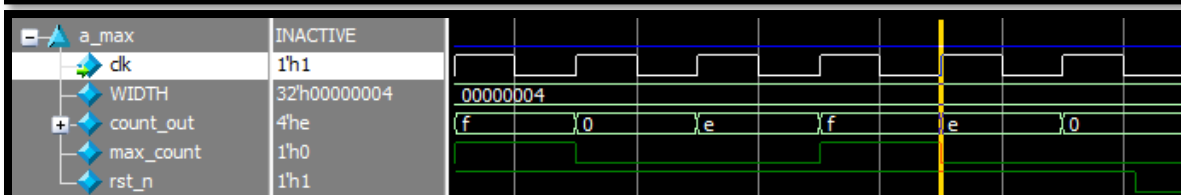
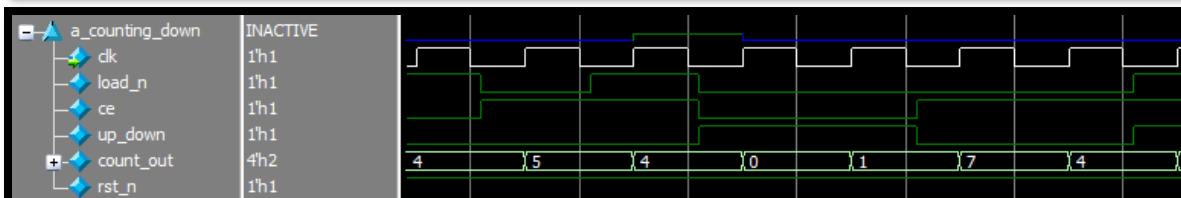
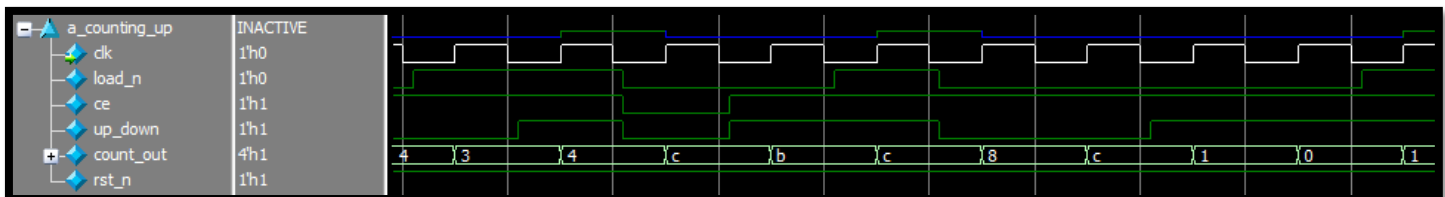
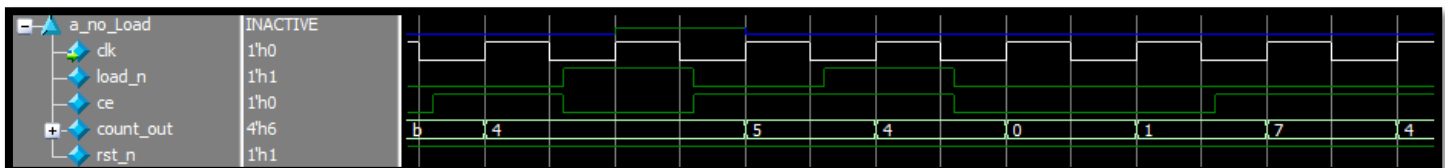
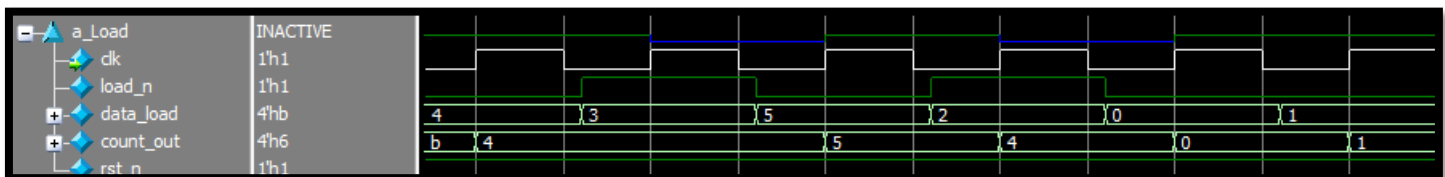
-Assertion Coverage:

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Count	Status
/counter_top/dut/counter_sva_instance/a_load_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(45)	588	Covered
/counter_top/dut/counter_sva_instance/a_no_load_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(48)	71	Covered
/counter_top/dut/counter_sva_instance/a_counting_up_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(51)	84	Covered
/counter_top/dut/counter_sva_instance/a_counting_down_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(54)	72	Covered
/counter_top/dut/counter_sva_instance/a_max_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(57)	45	Covered
/counter_top/dut/counter_sva_instance/a_zero_cover	counter_sva	Verilog	SVA	../../../../VS_code/Session4/Assignment4/Q3_Counter/Counter_sva.sv(60)	153	Covered

TOTAL DIRECTIVE COVERAGE: 100.0% COVERS: 6

-Waveforms:



Question4: Buggy Config Reg

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Reset	When the reset is asserted high, all the registers should be assigned their designated reset values set	-Directed at the start of the simulation -After writing all ones to all registers	assert_reset() -A checker which is an associative array "reset_assoc" containing the reset values to be checked against -Looping across all registers to check
Write	When Write is asserted, the set address should contain what data_in assigned	-Loop for all 8 addresses -Write All ones -Have a static array of hot one values for data_in to be written -Loop 64 times with random data_in values	check_result() A checker in the testbench to make sure the data_out = data_in before oncoming stimulus

-Testbench:

```
module config_reg_tb;

typedef enum {adc0_reg, adc1_reg, temp_sensor0_reg, temp_sensor1_reg,
analog_test, digital_test, amp_gain, digital_config} address_e;

reg clk, reset, write;
reg [15:0] data_in;
address_e address;
wire [15:0] data_out;

config_reg uut (.*);

integer correct_count = 0;
integer error_count = 0;

//Arrays and initialization=====
logic [15:0] reset_assoc[string];
initial begin
    reset_assoc["adc0_reg"]          = 16'hFFFF;
    reset_assoc["adc1_reg"]          = 16'h0000;
    reset_assoc["temp_sensor0_reg"]  = 16'h0000;
    reset_assoc["temp_sensor1_reg"]  = 16'h0000;
    reset_assoc["analog_test"]       = 16'hABCD;
    reset_assoc["digital_test"]       = 16'h0000;
    reset_assoc["amp_gain"]           = 16'h0000;
    reset_assoc["digital_config"]     = 16'h0001;
end

logic [15:0] hot_ones[ ] = '{
16'h0001, 16'h0002, 16'h0004, 16'h0008,
16'h0010, 16'h0020, 16'h0040, 16'h0080,
16'h0100, 16'h0200, 16'h0400, 16'h0800,
16'h1000, 16'h2000, 16'h4000, 16'h8000};
```

```

initial begin

    $display("=====
write = 0; reset = 0; data_in = 16'h0000;

    reset_and_check_all();

    //Test series2: Hot ones writing
write = 1; //writing data on DUT
@(negedge clk);
address = address.first;
for(i = 0 ; i < address.num ; i++) begin

    for (j = 0; j < hot_ones.size() ; j++)begin
        data_in = hot_ones[j];
        @(negedge clk);
        check_result();
    end

    address = address.next;
end
@(negedge clk);
reset_and_check_all();
@(negedge clk);
//Test series 3: Random Writing
write = 1; //writing data on DUT
@(negedge clk);
address = address.first;
for(i = 0 ; i < address.num ; i++) begin

    repeat(15) begin
        data_in = $random % 16'hFFFF;
        @(negedge clk);
        check_result();
    end

    address = address.next;
end
@(negedge clk);

```

```

//Test series 4: All High Writing
write = 1; //writing data on DUT
@(negedge clk);
address = address.first;
for(i = 0 ; i < address.num ; i++) begin

    repeat(1) begin
        data_in = 16'hFFFF;
        @(negedge clk);
        check_result();
    end

    address = address.next;
end
@(negedge clk);
reset_and_check_all();

```

-Bug Report:

Bug #1: analog_test	
Design i/p	Reset asserted
Expected	analog_test reset to 0xabcd
Observed	analog_test reset to 0xabcc

Bug #2: adc0_reg	
Design i/p	Write 0x0001 to adc0_reg
Expected	data_out = 0x0001 after 1 cycle
Observed	data_out = 0x8001 after 1 cycle

Bug #3: adc1_reg	
Design i/p	Write 0x0004 to adc0_reg
Expected	data_out = 0x0004 after 1 cycle
Observed	data_out = 0x0400 after 1 cycle (shifted by 8)

Bug #4: temp_sensor0_reg	
Design i/p	Write 0x0008 to temp_sensor0_reg
Expected	data_out = 0x0008 after 1 cycle
Observed	data_out = 0x0010 after 1 cycle (shifted by 1)

Bug #5: digital_test	
Design i/p	Write 0xe06c to digital_test
Expected	data_out = 0xe06c after 1 cycle
Observed	data_out = 0x8000 after 1 cycle (stuck at output)

Bug #6: digital_config	
Design i/p	Write 0xffff to digital_test
Expected	data_out = 0xffff after 1 cycle
Observed	data_out = 0x7fff after 1 cycle (MSB unwritable)

Bug #7: temp_sensor1_reg	
Design i/p	Reset asserted after writing 0xffff
Expected	data_out = 0x0000 after 1 cycle
Observed	data_out = 0xffff after 1 cycle

Bug #8: amp_gain	
Design i/p	Write 0x0020 to amp_gain
Expected	data_out = 0x0020 after 1 cycle
Observed	data_out = 0x8000 after 1 cycle (stuck at output)

